

Knapsack Problem

Wykonane przez studentów DSW: Jakub Buzuk dsw49445 i Maksym Belilovskyi
49438

Github:

<https://github.com/Baz00k/DSW> genetic algorithms and neural networks

W dokumencie ReadMe mamy instrukcje do odpalenia programu. Po instalowaniu wszystkich niezbędnych moduły Pythonu oraz stworzenia wirtualnego otoczenia za pomocą komendy:

```
python3 -m venv .venv
source .venv/bin/activate
```

Możemy uruchomić skrypt dla działania programu:

```
python -m knapsack_cli --population_size=100 --generations=100 --
mutation_rate=0.001 --input_file_name=low-dimensional/f10_1-
d kp 20 879
```

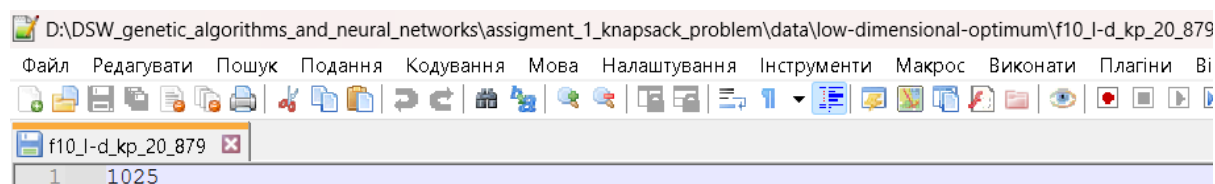
Dodamy do naszego kodu parametr `--enable_plots true`, żeby mieć możliwość zobaczyć grafik, opisujący poszukiwanie najlepszego rozwiązania:

1). low-dimensional/f10 l-d kp 20 879

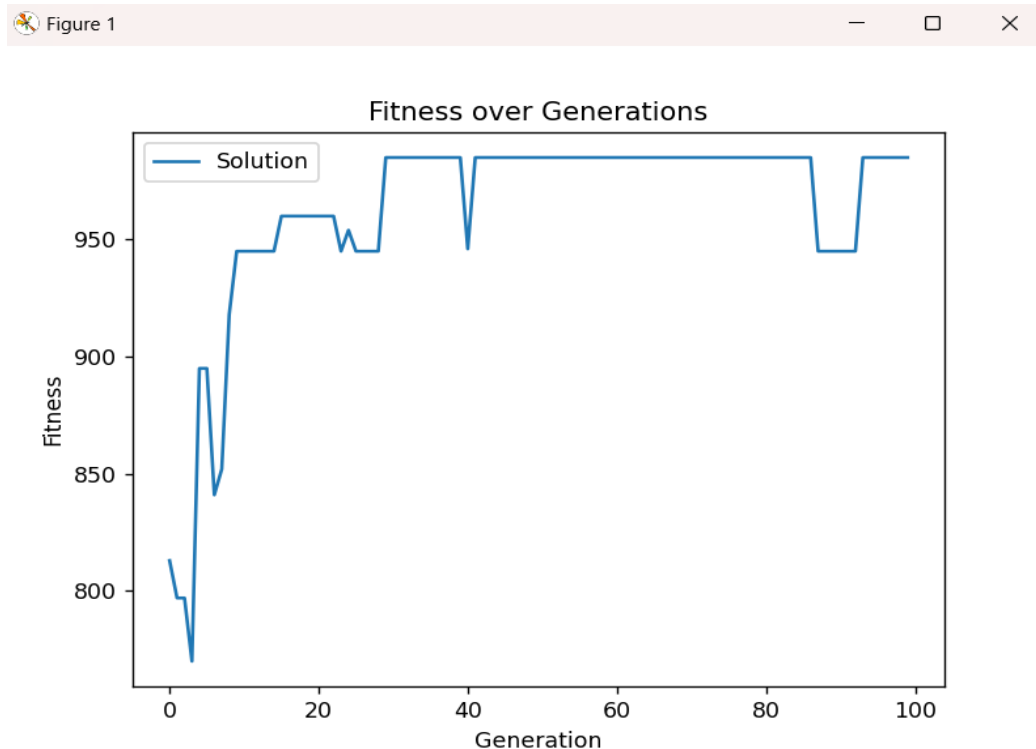
```
Lord@LAPTOP-LKTC6CJ6 MINGW64 /d/DSW_genetic_algorithms_and_neural_networks/assignment_1_knapsack_problem (development)
$ python -m knapsack_cli --population_size=100 --generations=100 --mutation_rate=0.001 --input_file_name=low-dimensional/f10_l-d_kp_20_879 --enable_plots true
Solving knapsack problem with 20 items and a capacity of 879
Using settings:
Population size: 100
Generations: 100
Crossover type: single_point
Mutation rate: 0.001
Selection type: roulette_wheel
Fitness type: linear

100%|██████████████████████████████████████████████████████████████████████████████| 100/100 [00:00<00:00, 161.60it/s]
Best solution found: Knapsack(total_weight=842, total_value=985, capacity=879)
(.venv)
```

Jak możemy zobaczyć, program pokazuje najlepsze rozwiązanie bliskie do optimum.



Za chwilę zobaczymy grafik



Na grafiku możemy zobaczyć proces znalezienia najlepszego rozwiązania przez algorytm.

#Możemy również włączyć opcje pokazywania grafików dla wszystkich sposobów selekcji (rankingowa, selekcyjna etc,) za pomocą komendy `--compare_solvers true`#

2).low_dimensional/f5_l-d_kp_15_375

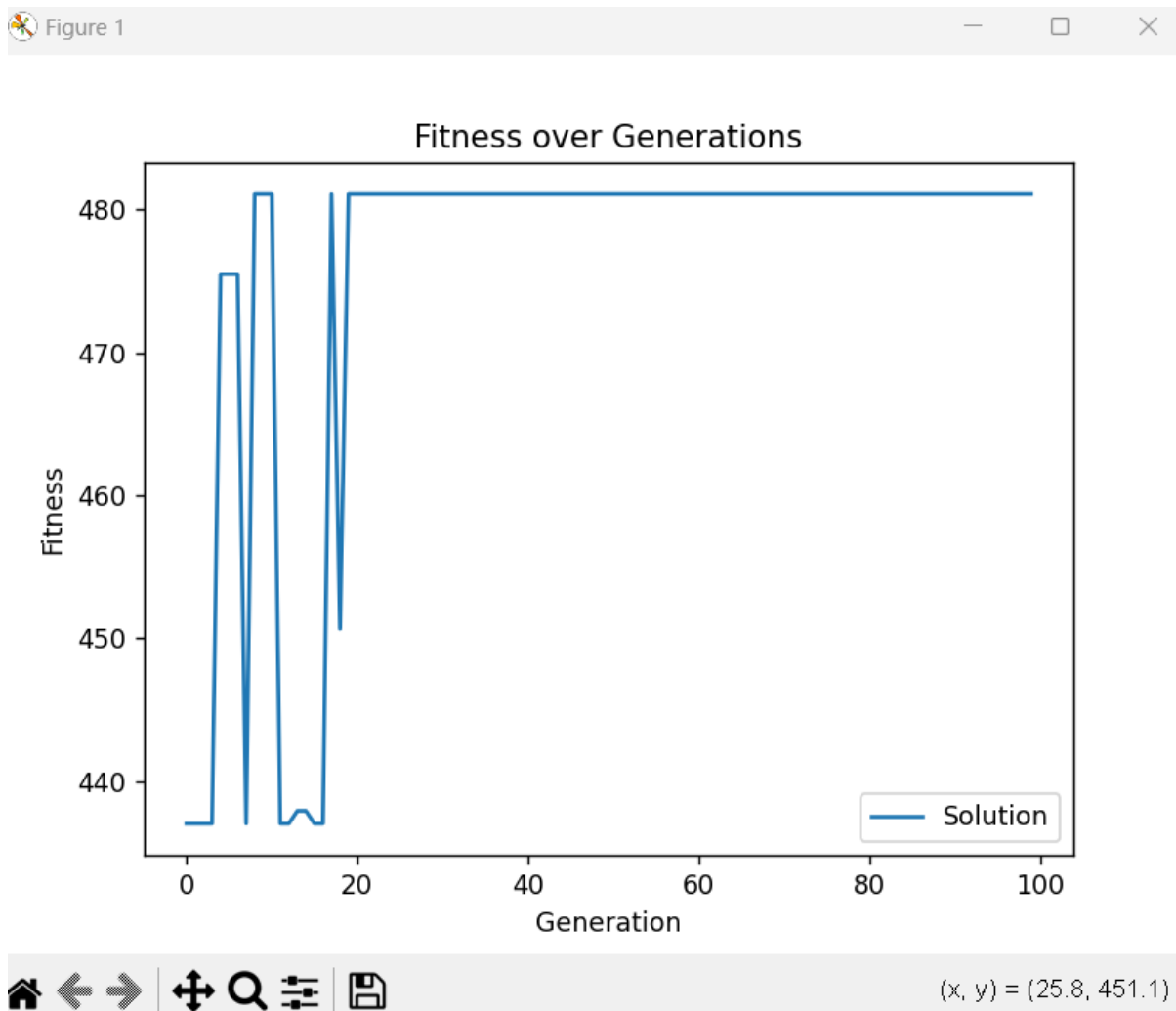
```

lord@LAPTOP-LKTC6CJ6 MINGW64 /d/bSw_genetic_algorithms_and_neural_networks/assignment_1_
knapsack_problem (development)
$ python -m knapsack_cli --population_size=100 --generations=100 --mutation_rate=0.001
--input_file_name=low-dimensional/f5_l-d_kp_15_375 --enable_plots true
solving knapsack problem with 15 items and a capacity of 375
using settings:
Population size: 100
Generations: 100
Crossover type: single_point
Mutation rate: 0.001
Selection type: roulette_wheel
Fitness type: linear

100%|████████████████████████████████████████████████████████████████████████████████| 100/100 [00:00<00:00, 168.68it/s]
Best solution found: Knapsack(total_weight=343.418684, total_value=431.708731, capacity=375)

```

Grafik:



Optimum:

```
D:\DSW_genetic_algorithms_and_neural_networks\assigment_1_knapsack_problem\data\low-dimensional-optimum\f5_I-d_kp_15_375
Файл Редагувати Пошук Подання Кодування Мова Налаштування Інструменти Макрос Виконати Плагіни В
f5_I-d_kp_15_375
1 481.0694
```

Możemy wywnioskować, że algorytm idealnie trafił w najlepsze rozwiązanie problemu.

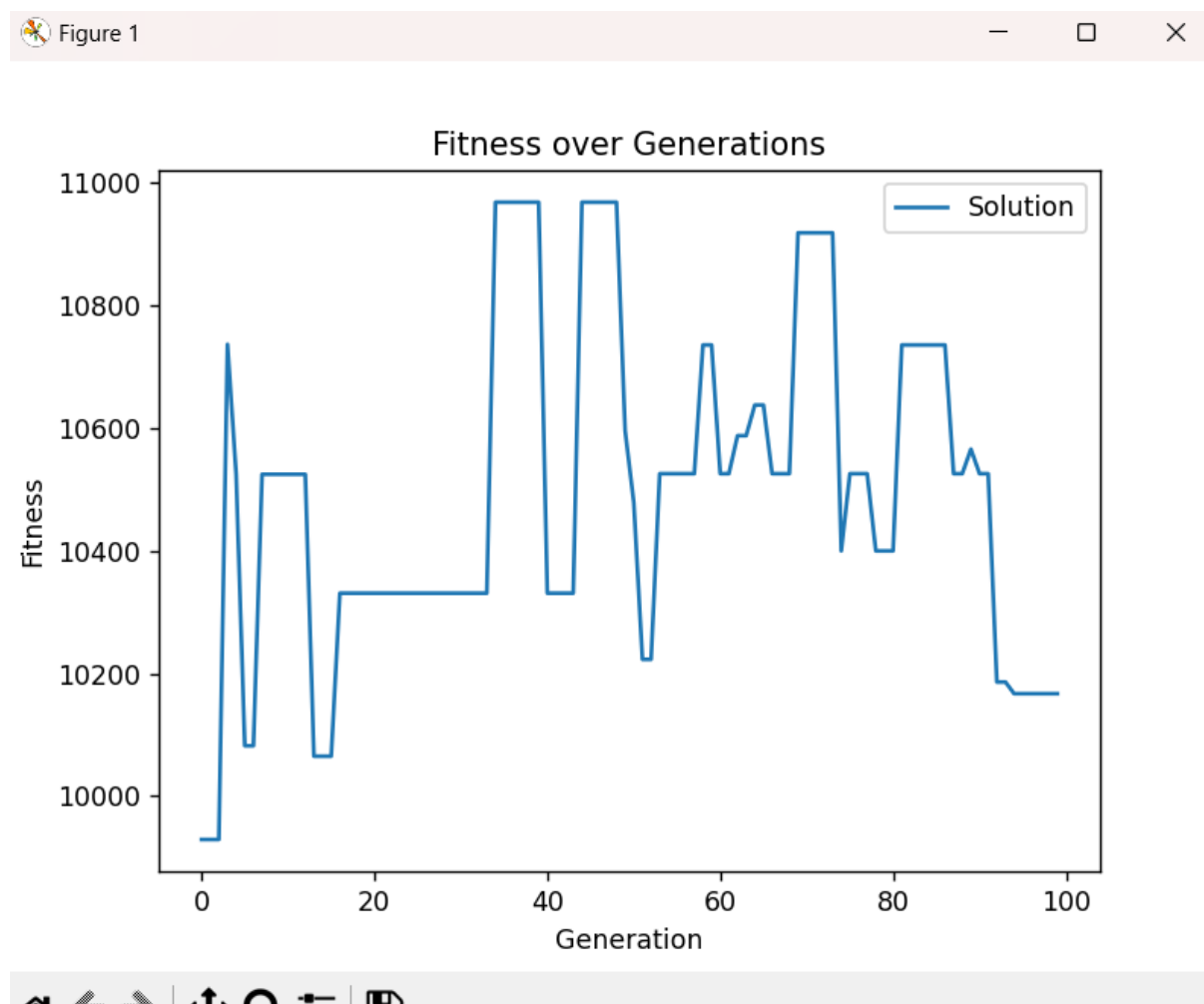
3). large_scale/knapPI_1_200_1000_1

```

Lord@LAPTOP-LKTC6CJ6 MINGW64 /d/dsw_genetic_algorithms_and_neural_networks/assignment_1
knapsack_problem (development)
$ python -m knapsack_cli --population_size=100 --generations=100 --mutation_rate=0.001
--input_file_name=large_scale/knapPI_1_200_1000_1 --enable_plots true
Solving knapsack problem with 200 items and a capacity of 1008
Using settings:
Population size: 100
Generations: 100
Crossover type: single_point
Mutation rate: 0.001
Selection type: roulette_wheel
Fitness type: linear
100%|████████████████████████████████████████████████████████████████████████████████| 100/100 [00:00<00:00, 113.25it/s]
Best solution found: knapsack(total_weight=972, total_value=10167, capacity=1008)

```

Wykres:



Optimum:

```

D:\DSW_genetic_algorithms_and_neural_networks\assignment_1_knapsack_problem\data\large_scale-optimum\knapPI_1_200_1000_1 - Notepad++
Файл  Редагувати  Пошук  Подання  Кодування  Мова  Налаштування  Інструменти  Макрос  Виконати  Плагіни  Вікно  ?
[Icons]
knapPI_1_2000_1000_1  knapPI_1_2000_1000_1  knapPI_1_200_1000_1
1  11238

```

Rozwiązanie było blisko do optimum

4). large_scale/knapPI_1_100_1000_1

```
Lord@LAPTOP-LKTC6CJ6 MINGW64 /d/DSW_genetic_algorithms_and_neural_networks/assignment_1
knapSack_problem (development)
$ python -m knapsack_cli --population_size=100 --generations=100 --mutation_rate=0.001
--input_file_name=large_scale/knapPI_1_100_1000_1 --enable_plots true
solving knapsack problem with 100 items and a capacity of 995
using settings:
Population size: 100
Generations: 100
Crossover type: single_point
Mutation rate: 0.001
selection type: roulette_wheel
Fitness type: linear
100%|████████████████████████████████████████████████████████████████████████████████| 100/100 [00:00<00:00, 137.18it/s]
Best solution found: knapsack(total_weight=968, total_value=8315, capacity=995)
```

Grafik:



Optimum

```
D:\DSW_genetic_algorithms_and_neural_networks\assignment_1_knapsack_problem\data\large_scale-optimum\knapPI_1_100_1000_1 - Notepad++
Файл Редагувати Пошук Подання Кодування Мова Налаштування Інструменти Макрос Виконати Плагіни Вікно ?
knapPI_1_2000_1000_1 knapPI_1_2000_1000_1 knapPI_1_200_1000_1 knapPI_1_100_1000_1
1 9147
```

Rozwiązanie było blisko do optimum

Wnioski: Dla zbiorów 'low_dimensional' będą słabe te wykresy, bo algorytm startuje z prawie optymalnego rozwiązania.

Dla zbiorów 'large_scale' algorytm pokazuje nieidealne rozwiązanie, bo po znalezieniu optimum zaczyna spadać. Możliwym rozwiązaniem jest wykorzystanie innego typu selekcji.